

NAME

ovn-northd – Open Virtual Network central control daemon

SYNOPSIS

ovn-northd [*options*]

DESCRIPTION

ovn-northd is a centralized daemon responsible for translating the high-level OVN configuration into logical configuration consumable by daemons such as **ovn-controller**. It translates the logical network configuration in terms of conventional network concepts, taken from the OVN Northbound Database (see **ovn-nb(5)**), into logical datapath flows in the OVN Southbound Database (see **ovn-sb(5)**) below it.

OPTIONS

--ovnnb-db=database

The OVSDB database containing the OVN Northbound Database. If the **OVN_NB_DB** environment variable is set, its value is used as the default. Otherwise, the default is **unix:/ovnnb_db.sock**.

--ovnsb-db=database

The OVSDB database containing the OVN Southbound Database. If the **OVN_SB_DB** environment variable is set, its value is used as the default. Otherwise, the default is **unix:/ovnsb_db.sock**.

--dry-run

Causes **ovn-northd** to start paused. In the paused state, **ovn-northd** does not apply any changes to the databases, although it continues to monitor them. For more information, see the **pause** command, under **Runtime Management Commands** below.

--n-threads=N

In certain situations, it may be desirable to enable parallelization on a system to decrease latency (at the potential cost of increasing CPU usage).

This option will cause ovn-northd to use N threads when building logical flows, when N is within [2–256]. If N is 1, parallelization is disabled (default behavior). If N is less than 1, then N is set to 1, parallelization is disabled and a warning is logged. If N is more than 256, then N is set to 256, parallelization is enabled (with 256 threads) and a warning is logged.

--dump-inc-proc-graph[=node]

Dump the incremental processing engine graph representation in DOT format to stdout at startup and then exit. If *node* is specified, only the subgraph rooted at that engine node is printed.

database in the above options must be an OVSDB active or passive connection method, as described in **ovsdb(7)**.

Daemon Options

--pidfile[=pidfile]

Causes a file (by default, *program.pid*) to be created indicating the PID of the running process. If the *pidfile* argument is not specified, or if it does not begin with /, then it is created in .

If **--pidfile** is not specified, no pidfile is created.

--overwrite-pidfile

By default, when **--pidfile** is specified and the specified pidfile already exists and is locked by a running process, the daemon refuses to start. Specify **--overwrite-pidfile** to cause it to instead overwrite the pidfile.

When **--pidfile** is not specified, this option has no effect.

--detach

Runs this program as a background process. The process forks, and in the child it starts a new session, closes the standard file descriptors (which has the side effect of disabling logging to the console), and changes its current directory to the root (unless **--no-chdir** is specified). After the child completes its initialization, the parent exits.

—monitor

Creates an additional process to monitor this program. If it dies due to a signal that indicates a programming error (**SIGABRT**, **SIGALRM**, **SIGBUS**, **SIGFPE**, **SIGILL**, **SIGPIPE**, **SIGSEGV**, **SIGXCPU**, or **SIGXFSZ**) then the monitor process starts a new copy of it. If the daemon dies or exits for another reason, the monitor process exits.

This option is normally used with **—detach**, but it also functions without it.

—no-chdir

By default, when **—detach** is specified, the daemon changes its current working directory to the root directory after it detaches. Otherwise, invoking the daemon from a carelessly chosen directory would prevent the administrator from unmounting the file system that holds that directory.

Specifying **—no-chdir** suppresses this behavior, preventing the daemon from changing its current working directory. This may be useful for collecting core files, since it is common behavior to write core dumps into the current working directory and the root directory is not a good directory to use.

This option has no effect when **—detach** is not specified.

—no-self-confinement

By default this daemon will try to self-confine itself to work with files under well-known directories determined at build time. It is better to stick with this default behavior and not to use this flag unless some other Access Control is used to confine daemon. Note that in contrast to other access control implementations that are typically enforced from kernel-space (e.g. DAC or MAC), self-confinement is imposed from the user-space daemon itself and hence should not be considered as a full confinement strategy, but instead should be viewed as an additional layer of security.

—user=user:group

Causes this program to run as a different user specified in *user:group*, thus dropping most of the root privileges. Short forms *user* and *:group* are also allowed, with current user or group assumed, respectively. Only daemons started by the root user accepts this argument.

On Linux, daemons will be granted **CAP_IPC_LOCK** and **CAP_NET_BIND_SERVICES** before dropping root privileges. Daemons that interact with a datapath, such as **ovs-vswitchd**, will be granted three additional capabilities, namely **CAP_NET_ADMIN**, **CAP_NET_BROADCAST** and **CAP_NET_RAW**. The capability change will apply even if the new user is root.

On Windows, this option is not currently supported. For security reasons, specifying this option will cause the daemon process not to start.

Logging Options

—v[spec]

—verbose=[spec]

Sets logging levels. Without any *spec*, sets the log level for every module and destination to **dbg**. Otherwise, *spec* is a list of words separated by spaces or commas or colons, up to one from each category below:

- A valid module name, as displayed by the **vlog/list** command on **ovs-appctl(8)**, limits the log level change to the specified module.
- **syslog**, **console**, or **file**, to limit the log level change to only to the system log, to the console, or to a file, respectively. (If **—detach** is specified, the daemon closes its standard file descriptors, so logging to the console will have no effect.)

On Windows platform, **syslog** is accepted as a word and is only useful along with the **—syslog-target** option (the word has no effect otherwise).

- **off**, **emer**, **err**, **warn**, **info**, or **dbg**, to control the log level. Messages of the given severity or higher will be logged, and messages of lower severity will be filtered out. **off** filters out all messages. See **ovs-appctl(8)** for a definition of each log level.

Case is not significant within *spec*.

Regardless of the log levels set for **file**, logging to a file will not take place unless **--log-file** is also specified (see below).

For compatibility with older versions of OVS, **any** is accepted as a word but has no effect.

-v

--verbose

Sets the maximum logging verbosity level, equivalent to **--verbose=dbg**.

-vPATTERN:destination:pattern

--verbose=PATTERN:destination:pattern

Sets the log pattern for *destination* to *pattern*. Refer to **ovs-appctl(8)** for a description of the valid syntax for *pattern*.

-vFACILITY:facility

--verbose=FACILITY:facility

Sets the RFC5424 facility of the log message. *facility* can be one of **kern**, **user**, **mail**, **daemon**, **auth**, **syslog**, **lpr**, **news**, **uucp**, **clock**, **ftp**, **ntp**, **audit**, **alert**, **clock2**, **local0**, **local1**, **local2**, **local3**, **local4**, **local5**, **local6** or **local7**. If this option is not specified, **daemon** is used as the default for the local system syslog and **local0** is used while sending a message to the target provided via the **--syslog-target** option.

--log-file[=file]

Enables logging to a file. If *file* is specified, then it is used as the exact name for the log file. The default log file name used if *file* is omitted is **/usr/local/var/log/ovn/program.log**.

--syslog-target=host:port

Send syslog messages to UDP *port* on *host*, in addition to the system syslog. The *host* must be a numerical IP address, not a hostname.

--syslog-method=method

Specify *method* as how syslog messages should be sent to syslog daemon. The following forms are supported:

- **libc**, to use the libc **syslog()** function. Downside of using this options is that libc adds fixed prefix to every message before it is actually sent to the syslog daemon over **/dev/log** UNIX domain socket.
- **unix:file**, to use a UNIX domain socket directly. It is possible to specify arbitrary message format with this option. However, **rsyslogd 8.9** and older versions use hard coded parser function anyway that limits UNIX domain socket use. If you want to use arbitrary message format with older **rsyslogd** versions, then use UDP socket to localhost IP address instead.
- **udp:ip:port**, to use a UDP socket. With this method it is possible to use arbitrary message format also with older **rsyslogd**. When sending syslog messages over UDP socket extra precaution needs to be taken into account, for example, syslog daemon needs to be configured to listen on the specified UDP port, accidental iptables rules could be interfering with local syslog traffic and there are some security considerations that apply to UDP sockets, but do not apply to UNIX domain sockets.
- **null**, to discard all messages logged to syslog.

The default is taken from the **OVS_SYSLOG_METHOD** environment variable; if it is unset, the default is **libc**.

PKI Options

PKI configuration is required in order to use SSL/TLS for the connections to the Northbound and Southbound databases.

- p** *privkey.pem*
- private-key=***privkey.pem*
Specifies a PEM file containing the private key used as identity for outgoing SSL/TLS connections.
- c** *cert.pem*
- certificate=***cert.pem*
Specifies a PEM file containing a certificate that certifies the private key specified on **-p** or **--private-key** to be trustworthy. The certificate must be signed by the certificate authority (CA) that the peer in SSL/TLS connections will use to verify it.
- C** *cacert.pem*
- ca-cert=***cacert.pem*
Specifies a PEM file containing the CA certificate for verifying certificates presented to this program by SSL/TLS peers. (This may be the same certificate that SSL/TLS peers use to verify the certificate specified on **-c** or **--certificate**, or it may be a different one, depending on the PKI design in use.)
- C** **none**
- ca-cert=****none**
Disables verification of certificates presented by SSL/TLS peers. This introduces a security risk, because it means that certificates cannot be verified to be those of known trusted hosts.
- ssl-server-name=***servername*
Specifies the server name to use for TLS Server Name Indication (SNI). By default, the hostname from the connection string is used for SNI. This option allows overriding the SNI hostname, which is useful when connecting through proxies or service meshes where the connection endpoint differs from the intended server name.

Other Options

- unixctl=***socket*
Sets the name of the control socket on which *program* listens for runtime management commands (see *RUNTIME MANAGEMENT COMMANDS*, below). If *socket* does not begin with */*, it is interpreted as relative to *.* If **--unixctl** is not used at all, the default socket is */program.pid.ctl*, where *pid* is *program*'s process ID.

On Windows a local named pipe is used to listen for runtime management commands. A file is created in the absolute path as pointed by *socket* or if **--unixctl** is not used at all, a file is created as *program* in the configured *OVS_RUNDIR* directory. The file exists just to mimic the behavior of a Unix domain socket.

Specifying **none** for *socket* disables the control socket feature.

- h**
- help** Prints a brief help message to the console.
- V**
- version**
Prints version information to the console.

RUNTIME MANAGEMENT COMMANDS

ovn-appctl can send commands to a running **ovn-northd** process. The currently supported commands are described below.

- exit** Causes **ovn-northd** to gracefully terminate.
- pause** Pauses **ovn-northd**. When it is paused, **ovn-northd** receives changes from the North-bound and Southbound database changes as usual, but it does not send any updates. A paused **ovn-northd** also drops database locks, which allows any other non-paused

instance of **ovn-northd** to take over.

resume Resumes the ovn-northd operation to process Northbound and Southbound database contents and generate logical flows. This will also instruct ovn-northd to aspire for the lock on SB DB.

is-paused

Returns "true" if ovn-northd is currently paused, "false" otherwise.

status Prints this server's status. Status will be "active" if ovn-northd has acquired OVSDB lock on SB DB, "standby" if it has not or "paused" if this instance is paused.

sb-cluster-state-reset

Reset southbound database cluster status when databases are destroyed and rebuilt.

If all databases in a clustered southbound database are removed from disk, then the stored index of all databases will be reset to zero. This will cause ovn-northd to be unable to read or write to the southbound database, because it will always detect the data as stale. In such a case, run this command so that ovn-northd will reset its local index so that it can interact with the southbound database again.

nb-cluster-state-reset

Reset northbound database cluster status when databases are destroyed and rebuilt.

This performs the same task as **sb-cluster-state-reset** except for the northbound database client.

parallel-build/set-n-threads *N*

Set the number of threads used for building logical flows. When *N* is within [2–256], parallelization is enabled. When *N* is 1 parallelization is disabled. When *N* is less than 1 or more than 256, an error is returned. If ovn-northd fails to start parallelization (e.g. fails to setup semaphores), parallelization is disabled and an error is returned.

parallel-build/get-n-threads

Return the number of threads used for building logical flows.

nb-connection-status

Prints whether the connection to the OVN Northbound database is currently connected or not.

sb-connection-status

Prints whether the connection to the OVN Southbound database is currently connected or not.

debug/chassis-features-list

Lists the chassis feature flags and their current values as determined by **ovn-northd**. These features reflect the capabilities reported by the connected chassis.

inc-engine/show-stats

Display **ovn-northd** engine counters. For each engine node the following counters have been added:

- **recompute**
- **compute**
- **abort**

inc-engine/show-stats *engine_node_name* [*counter_name*]

Display the **ovn-northd** engine counter(s) for the specified *engine_node_name*. *counter_name* is optional and can be one of **recompute**, **compute** or **abort**.

inc-engine/clear-stats

Reset **ovn-northd** engine counters.

inc-engine/recompute

Triggers a full recompute of the incremental processing engine on the next iteration.

inc-engine/compute-log-timeout *msecs*

Sets the timeout in milliseconds for logging engine compute events.

inc-engine/list-stopwatches [*node*]

Lists the engine nodes and their change handler names. If *node* is specified, only the handlers for that engine node are listed.

ACTIVE-STANDBY FOR HIGH AVAILABILITY

You may run **ovn-northd** more than once in an OVN deployment. When connected to a standalone or clustered DB setup, OVN will automatically ensure that only one of them is active at a time. If multiple instances of **ovn-northd** are running and the active **ovn-northd** fails, one of the hot standby instances of **ovn-northd** will automatically take over.

Active-Standby with multiple OVN DB servers

You may run multiple OVN DB servers in an OVN deployment with:

- OVN DB servers deployed in active/passive mode with one active and multiple passive ovsdb-servers.
- **ovn-northd** also deployed on all these nodes, using unix ctl sockets to connect to the local OVN DB servers.

In such deployments, the ovn-northds on the passive nodes will process the DB changes and compute logical flows to be thrown out later, because write transactions are not allowed by the passive ovsdb-servers. It results in unnecessary CPU usage.

With the help of runtime management command **pause**, you can pause **ovn-northd** on these nodes. When a passive node becomes master, you can use the runtime management command **resume** to resume the **ovn-northd** to process the DB changes.

LOGICAL FLOW TABLE STRUCTURE

For a detailed description of the logical flow tables that **ovn-northd** populates in the **OVN_Southbound** database, including logical switch and router datapath pipelines and drop sampling behavior, see **ovn-logical-flows(7)**.